PrivAgent: Plain-English Interview Defense Guide

Author: Yash Thakre (Roll No: 23BTB0A33 | NIT Warangal) | **Stack:** LangGraph, Semantic Kernel, MCP, Azure OpenAI

1. The Core Idea in Simple Words (Restaurant Analogy)

Imagine a busy restaurant:

• **Standard AI (ChatGPT):** Like one waiter who tries to cook, clean, serve, and bill all alone. He gets overwhelmed and makes up orders (hallucinates).

• **PrivAgent:** Like an entire coordinated restaurant team:

1. **Security Guard (Security Agent):** Checks role permissions and stops dangerous requests (e.g. DROP TABLE).
2. **Receptionist (Intent Agent):** Welcomes user and picks the fastest path (fast cache vs deep search).
3. **Manager (Planning Agent):** Breaks big goals into small tasks for specialist worker agents.
4. **Specialist Chefs (Investigator Agents):** SQL Chef (DB), Docs Chef (PDF RAG), Code Chef (Git AST), Ticket Chef (Jira).
5. **Head Chef (Analyst Agent):** Combines all findings into a clean, structured Markdown response.
6. **Quality Inspector (Critic Agent):** Audits the draft against raw logs before serving. If wrong, sends it back to fix!
7. **Owner Approval (Human-in-the-Loop):** Pauses for admin sign-off before modifying or deleting data.

2. How to Introduce Your Project (Say This Out Loud)

"Hi! For my project, I built PrivAgent—a Private Enterprise AI Operating System.

The problem is that enterprises have confidential data across SQL databases, PDFs, Jira tickets, and code. They cannot send this to public cloud AI due to strict privacy regulations (GDPR/HIPAA).

I used LangGraph to build a cyclic multi-agent team running on local hardware. The system plans tasks, queries specialized tools, audits drafts through a Critic to stop hallucinations, and delivers verified answers.

I also added a Semantic Kernel hybrid gateway that handles 90% of routine queries on free local models and only falls back to Azure OpenAI (GPT-4o) when needed, cutting cloud inference costs by 90%."

3. File-by-File Breakdown (What Each File Does)

File Name	Role & Purpose (Simple Words)
src/state.py	The **Shared Notebook** holding user query, plan, logs, and draft response.
src/graph.py	The **Traffic Controller** connecting 13 agent nodes in LangGraph.
src/agents/router.py	Gateway containing Security (RBAC), Intent (Cache), and Planning agents.
src/agents/investigators.py	Worker agents for SQL DB, Hybrid RAG Docs, Code AST, and Jira tickets.
src/agents/synthesizer.py	Analyst (Writer), Critic (Auditor), and Human Approval (HITL) nodes.

src/sk_router.py	Semantic Kernel Gateway: Local Ollama primary + Azure OpenAI fallback.
src/mcp_tools.py	Model Context Protocol: Standardizes tools into JSON-RPC compliance.
src/api.py	FastAPI web backend with PDF text and Vision OCR fallback.

4. Key Concepts Explained in Simple Terms

• **Hybrid RAG (Dense + BM25):** Vector embeddings understand general meaning, while BM25 finds exact numbers/names (like *23BTB0A33* or *JIRA-409*). Combining 70% vector + 30% BM25 gives 100% accurate search.

• **Critic Hallucination Filter:** Checks the Analyst draft word-by-word against raw retrieved logs. If an entity is not in the logs, the Critic flags it and loops back (up to 2 retries).

• **90% Cost Reduction:** Local Ollama runs on internal hardware for . Only 10% heavy queries route to Azure OpenAI, saving 90% of token bills.

• **Human-in-the-Loop (HITL):** Any write action (UPDATE, DELETE) freezes the graph execution and requires explicit human admin confirmation.

5. Top 3 Interview Questions & Direct Answers

Q1: Why LangGraph instead of standard LangChain?

“LangChain is linear (A to B to C). If step B fails, it crashes. LangGraph supports cyclic graphs, so our Critic can loop back to the Analyst, the Planner can replan on error, and the graph can pause for human approval.”

Q2: What is Model Context Protocol (MCP)?

“MCP is an open standard that acts like a universal USB connector for AI tools. It uses JSON-RPC so agents can talk to databases, Jira, or code without messy custom code.”

Q3: What were your biggest technical challenges?

“1. Small local models misclassifying tasks (solved with a rule-based Plan Validator). 2. Multi-agent logs overflowing the context window (solved with log_groomer). 3. LLM hallucinations on missing data (solved with the Critic verification loop).”